



PROGRAMAÇÃO



Escola Naval



António Gonçalves

Agenda – Funções I

1 Definição e chamadas

Executar e reutilizar instruções.

Definir uma
função



Chamar
a função

Uma função é definida com um nome e um conjunto de instruções que podem ser executadas mais do que uma vez.

2 Parâmetros e argumentos

Comunicar dados à função.

Argumento
(valor)



Função

Parâmetro
(variável)

Os argumentos fornecem os valores ou expressões que são recebidos pelos parâmetros da função.

3 Resultados com **return**

Devolver e reutilizar valores.

Função
e instruções



return



Valor
devolvido

A instrução **return** devolve um valor que pode ser reutilizado no código que chamou a função.

4 Decomposição de problemas

Criar e testar pequenas funções.

Problema
complexo



Pequenas
funções



Solução
mais simples

Um problema pode ser dividido em pequenas funções, que são criadas e testadas individualmente.

FUNÇÕES E MODULARIZAÇÃO

Função: bloco de instruções com um nome e uma responsabilidade específica. Pode ser reutilizada em várias partes do programa através de chamadas.

Exemplos de funções existentes

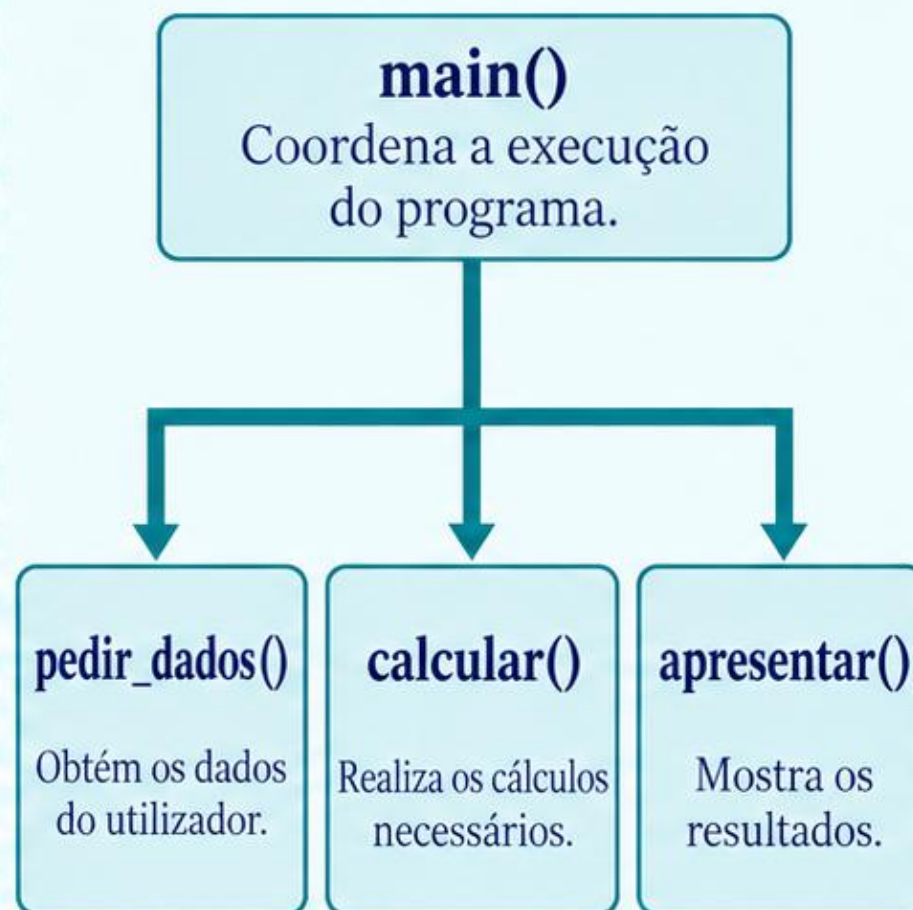
Já existem funções disponíveis que podemos utilizar, por exemplo:

```
print("Olá")
```

```
int("25")
```

Estas funções fazem parte da linguagem e estão integradas no interpretador.

Estrutura do programa



Funções que definimos

Além das funções integradas, podemos definir as nossas próprias funções para organizar o programa em partes lógicas.

```
def calcular():  
    # instruções da função  
    ...
```

Vantagens da modularização

- 1 Menos código repetido.
- 2 Programa mais claro e fácil de compreender.
- 3 Permite testar cada função separadamente.

Dividir o programa em funções torna-o mais organizado, reutilizável e fácil de manter.

DEFINIÇÃO E CHAMADA

def: palavra-chave para definir uma função.

Nome da função: identifica a função. Aqui: saudacao.

Parênteses (): podem conter parâmetros. Aqui, não há.

Dois pontos: indica o início do bloco de instruções da função.

```
def saudacao():  
    print("Bom dia!")
```

Corpo da função: instruções executadas quando a função é chamada. Está indentado.

```
saudacao()  
saudacao()
```

Chamadas da função: executam o corpo da função. Aqui, a função é chamada duas vezes.

- 1 A definição da função torna-a disponível para ser chamada.
- 2 O corpo da função só é executado quando a função é chamada.
- 3 Neste exemplo, há uma definição e duas chamadas.

Saída

```
Bom dia!  
Bom dia!
```

Definimos a função uma vez e podemos chamá-la sempre que quisermos.

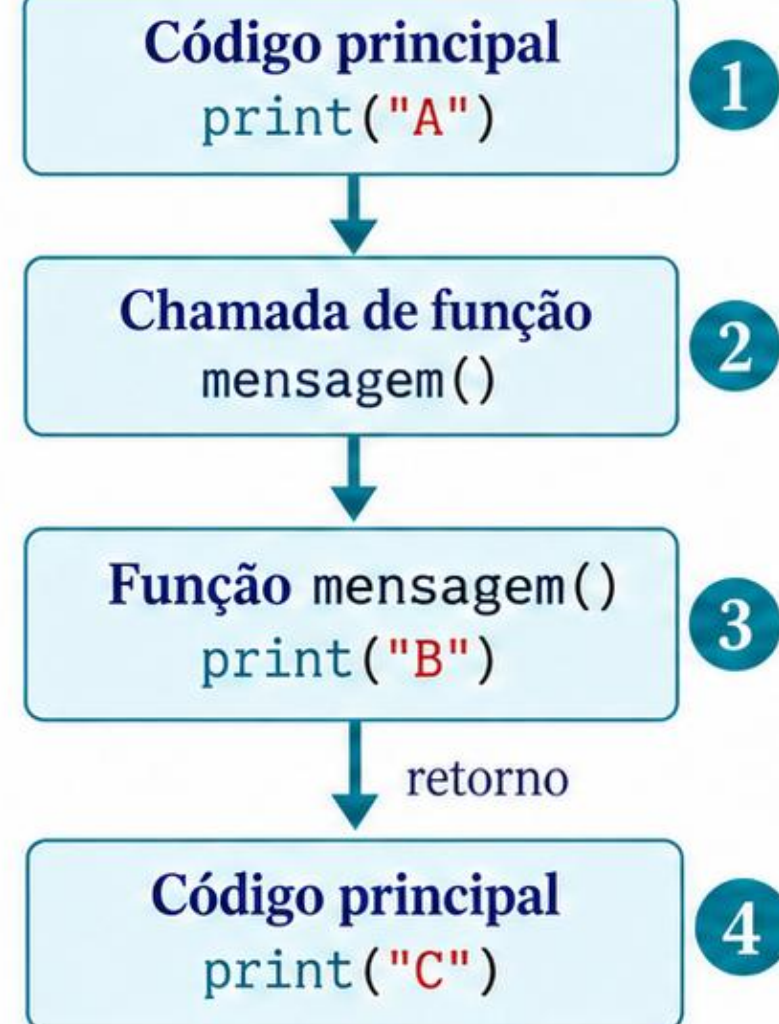
FLUXO DE EXECUÇÃO

Código

```
def mensagem():  
    print("B")  
  
print("A")  
mensagem()  
print("C")
```

- 1 Executa `print("A")`.
É apresentado A.
- 2 Chama a função `mensagem()`.
O controlo é transferido para o corpo da função.
- 3 Executa `print("B")` dentro da função.
É apresentado B.
Termina a função e o controlo regressa ao ponto da chamada.
- 4 Executa `print("C")`.
É apresentado C.

Fluxo de execução



Saída no ecrã

A
B
C

A definição da função `mensagem()` não executa o seu corpo. O `print("B")` só é executado quando a função é chamada.

Durante uma chamada de função, o controlo passa para o corpo da função e, quando termina, regressa ao ponto da chamada, continuando a execução.

PARÂMETROS E ARGUMENTOS

```
def mostrar_dobro(numero):  
    print(numero * 2)
```

```
mostrar_dobro(5)  
valor = 7  
mostrar_dobro(valor)  
mostrar_dobro(3 + 4)
```

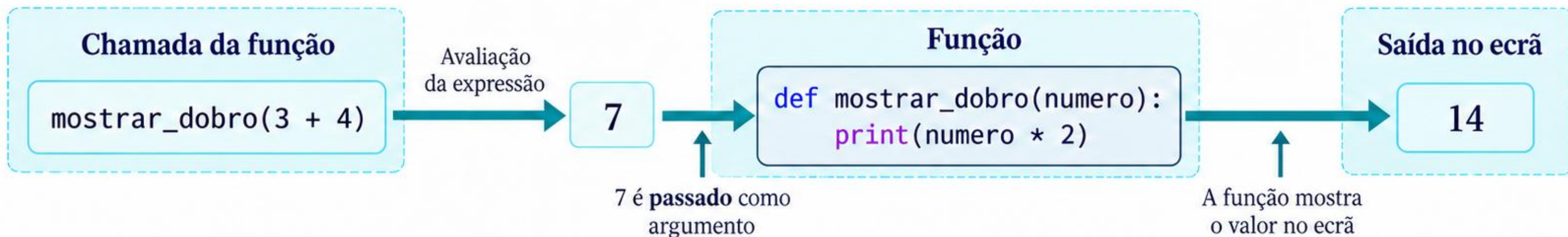
Parâmetro:

é o nome (variável) definido na função para receber um valor.

Argumento:

é o valor ou expressão fornecido na chamada da função.

Chamada	Valor recebido por numero	Saída
mostrar_dobro(5)	5	10
mostrar_dobro(valor)	7	14
mostrar_dobro(3 + 4)	7	14



A expressão do argumento é avaliada **antes** da chamada da função.

O parâmetro **numero** é o nome usado na definição da função para receber o valor.

O argumento é o valor ou expressão fornecido na chamada da função.

Argumentos posicionais e nomeados

Uma função pode receber **argumentos posicionais** (pela ordem) ou **nomeados** (pelo nome do parâmetro).

```
def dividir(a, b):  
    return a / b
```

Assume-se que **b** é diferente de zero.

Argumentos posicionais
(pela ordem)

`dividir(10, 2)`

`a = 10`

`b = 2`

Resultado: 5.0

Argumentos posicionais
(pela ordem)

`dividir(2, 10)`

`a = 2`

`b = 10`

Resultado: 0.2

Argumentos nomeados
(pelo nome)

`dividir(b=2, a=10)`

`a = 10`

`b = 2`

Resultado: 5.0

Mistura de argumentos
(posicional e nomeado)

`dividir(10, b=2)`

`a = 10`

`b = 2`

Resultado: 5.0

Exemplo inválido

`dividir(a=10, 2)`

SyntaxError

- 1 Nos argumentos posicionais, os valores são atribuídos aos parâmetros pela ordem em que aparecem.
- 2 Nos argumentos nomeados, os valores são atribuídos pelo nome do parâmetro. É possível misturar argumentos posicionais e nomeados, desde que os posicionais apareçam primeiro.

Valores predefinidos

Código em Python

```
def aplicar_desconto(preco, taxa=0.10):  
    return preco * (1 - taxa)
```

```
print(aplicar_desconto(100))  
print(aplicar_desconto(100, 0.20))  
print(aplicar_desconto(100, taxa=0.05))
```

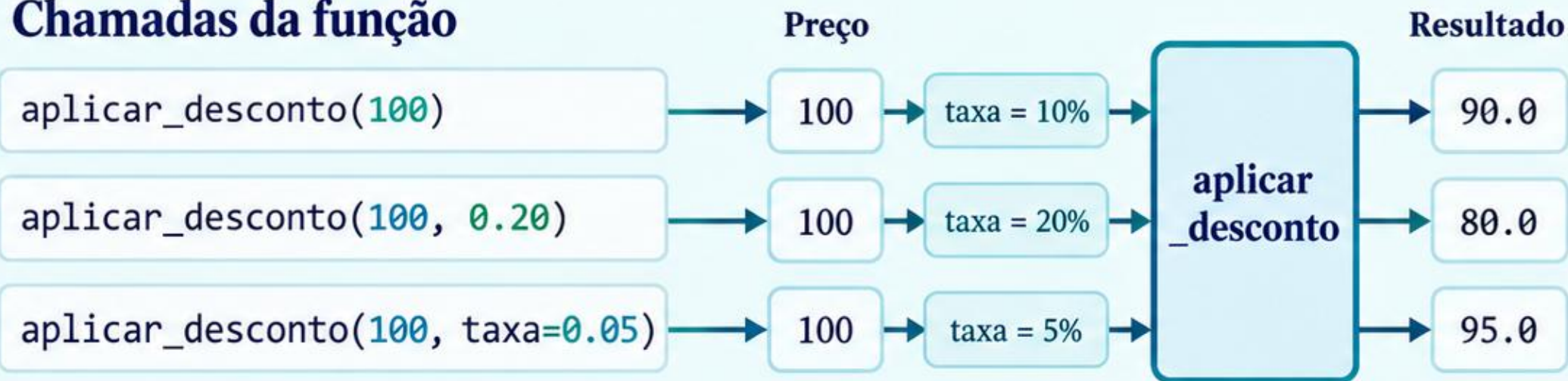
O valor predefinido (0.10) é utilizado quando o argumento é omitido na chamada da função. Um valor fornecido na chamada substitui o valor predefinido.

Na assinatura desta função, os parâmetros sem valores predefinidos (preco) aparecem antes dos parâmetros com valores predefinidos (taxa).

Saída

90.0
80.0
95.0

Chamadas da função



Se o argumento for omitido, utiliza-se o valor predefinido.
Se for fornecido um valor, este substitui o valor predefinido nessa chamada.

Erros nas chamadas

Definição da função

```
def somar(a, b):  
    return a + b
```

Parâmetros:

são os nomes das variáveis (a, b) definidos na função para receber os valores.

Chamada da função	Resultado	Observação
somar(2, 3)	5	Chamada correta.
somar(2)	TypeError: falta um argumento	Faltam argumentos.
somar(2, 3, 4)	TypeError: argumentos a mais	Foram fornecidos argumentos a mais.
somar(2, a=3)	TypeError: dois valores para a	O parâmetro a recebeu dois valores.

Exemplo de chamada correta

Chamada da função

somar(2, 3)

Valor devolvido

5

Testar chamadas separadamente

Cada chamada deve ser testada individualmente. Sem tratamento, um erro interrompe a execução do programa.

Lembre-se

- Use o nome exacto da função (somar).
- Utilize parênteses.
- Forneça o número correcto de argumentos.
- Respeite a ordem dos argumentos.

Valores devolvidos com return

Código em Python

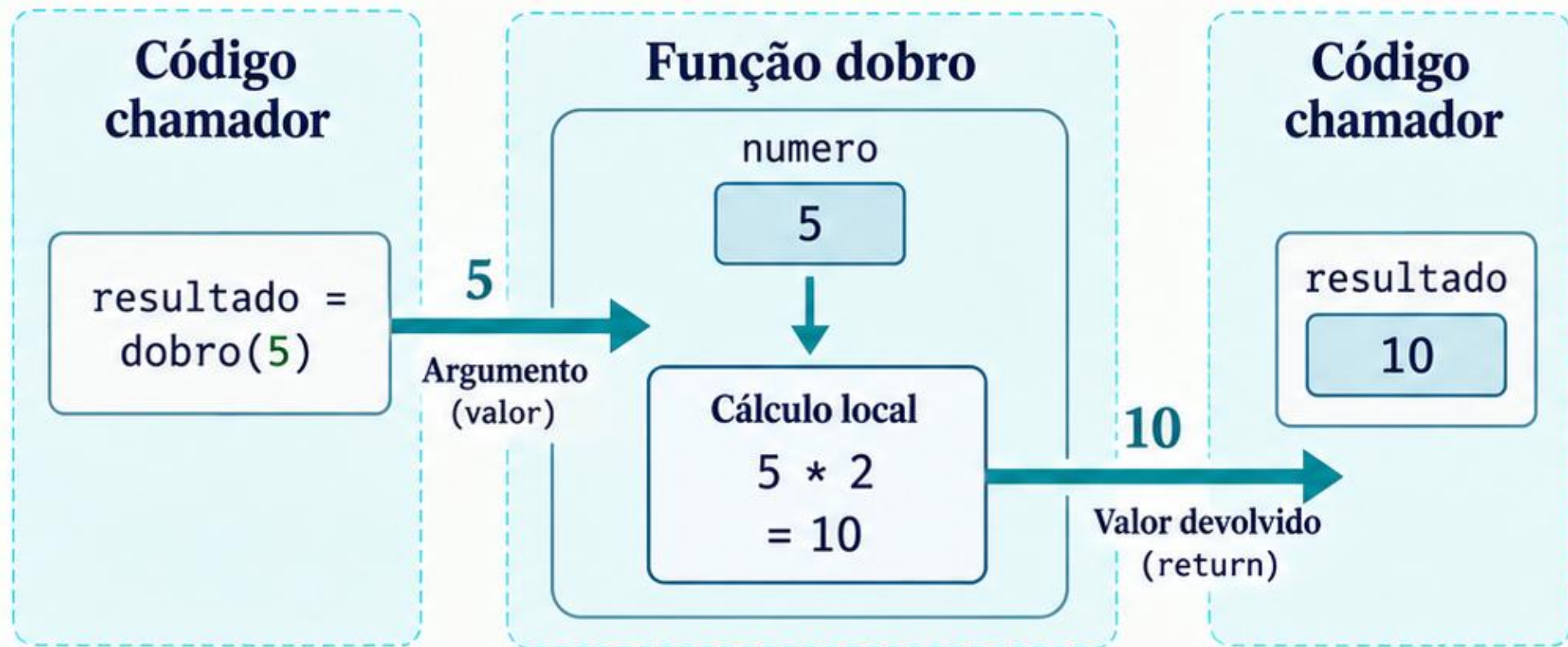
```
def dobro(numero):  
    return numero * 2  
  
resultado = dobro(5)  
print(resultado)  
print(dobro(5) + 3)
```

Saída

10

13

Como funciona a chamada de função



A instrução **return** envia o valor devolvido para o código que chamou a função e termina a chamada atual.

Se uma função chegar ao fim da sua execução sem instrução **return**, o valor devolvido é **None**.

print() e return

print()

Apresenta um valor no ecrã. Neste exemplo, mostrar_dobro devolve None.

```
def mostrar_dobro(n):  
    print(n * 2)  
  
r = mostrar_dobro(5)  
print(r)
```

Ecrã



Após a chamada

Variáveis do chamador

Nome	Valor
r	None

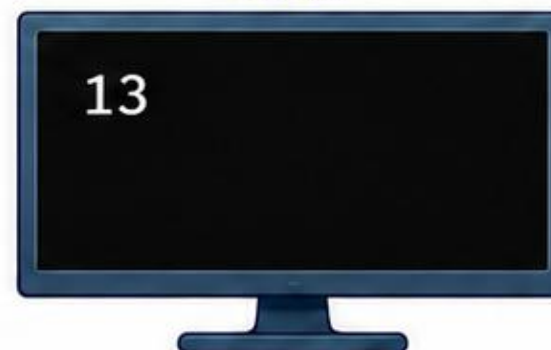
A função mostra o resultado (10) no ecrã, mas devolve **None**, que fica atribuído à variável **r**.

return

Devolve um valor que pode ser utilizado posteriormente no programa.

```
def calcular_dobro(n):  
    return n * 2  
  
r = calcular_dobro(5)  
print(r + 3)
```

Ecrã



Após a chamada

Variáveis do chamador

Nome	Valor
r	10

A função devolve o valor 10, que fica atribuído à variável **r** e pode ser usado em cálculos (**r + 3**).

print() apresenta resultados; return fornece um valor reutilizável.

TIPOS DE RESULTADO E DECISÕES

1 Resultados numéricos

Uma função pode devolver valores numéricos, como inteiros ou reais.

```
def media(a, b):  
    return (a + b) / 2
```

Chamada:

media(12, 16)

Resultado:

14.0
(float)

A função devolve a média dos dois valores, que é um número real do tipo **float**.

2 Resultados de texto

Uma função pode devolver cadeias de caracteres (strings).

```
def classificar(nota):  
    if nota >= 10:  
        return "Aprovado"  
    return "Reprovado"
```

Chamada:

classificar(14)

Resultado:

"Aprovado"
(str)

Assumindo que a nota está no intervalo 0..20, a função devolve uma cadeia de caracteres indicando o resultado.

3 Resultados lógicos e decisões

Uma função pode devolver valores lógicos, como **True** ou **False**.

```
def e_par(n):  
    return n % 2 == 0
```

Chamada:

e_par(8)

Resultado:

True
(bool)

Utilização numa decisão:

```
if e_par(8):  
    print("É par")
```



A instrução **return** pode devolver valores de vários tipos, incluindo **int**, **float**, **str**, **bool** e outros tipos.

return termina a chamada

Código em Python

```
def maior(a, b):  
    if a >= b:  
        return a  
    return b  
  
print(maior(8, 3))  
print(maior(2, 9))
```

Saída no ecrã

8
9

Função: é definida para receber dois valores e devolver o maior deles.

Instrução return: termina a chamada da função e devolve o valor indicado. A execução da função termina nesse ponto.

Avaliação das chamadas

Chamada	Valores de a e b	Resultado da condição a >= b	Instrução executada	Valor devolvido
maior(8, 3)	a = 8, b = 3	True	return a	8
maior(2, 9)	a = 2, b = 9	False	return b	9

Quando a condição é **True**, executa-se **return a** e o valor é devolvido. A instrução **return b** não chega a ser executada.

Quando a condição é **False**, executa-se **return b** e o valor é devolvido.

Ausência de return e valor None

Função sem return

```
def dobro(n):  
    n * 2
```

```
r = dobro(5)  
print(r)
```

Chamada da função

`r = dobro(5)` → **None**

A função termina e devolve None implicitamente.

Expressão sem return

Uma expressão sozinha, como `n * 2`, é avaliada, mas não devolve um valor da função.

Devolução implícita

Se uma função terminar sem instrução `return`, o valor devolvido é **None** implicitamente.

O valor None

None não é zero e não é a cadeia de caracteres **"None"**. É um valor especial que indica a ausência de um valor devolvido.

Função com return

```
def dobro(n):  
    return n * 2
```

```
r = dobro(5)  
print(r)
```

Chamada da função

`r = dobro(5)` → **10**

A função devolve o valor calculado.

Saída no ecrã

None

Saída no ecrã

10

Dados de entrada e resultado

Código em Python

```
def area_retangulo(base, altura):  
    return base * altura
```

```
b = float(input('Base: '))  
h = float(input('Altura: '))  
a = area_retangulo(b, h)  
print('Área:', a)
```

Função:

recebe dois valores (parâmetros), calcula a área do retângulo e devolve o resultado.

Parâmetros:

são os nomes (variáveis) definidos na função para receber os valores de entrada.

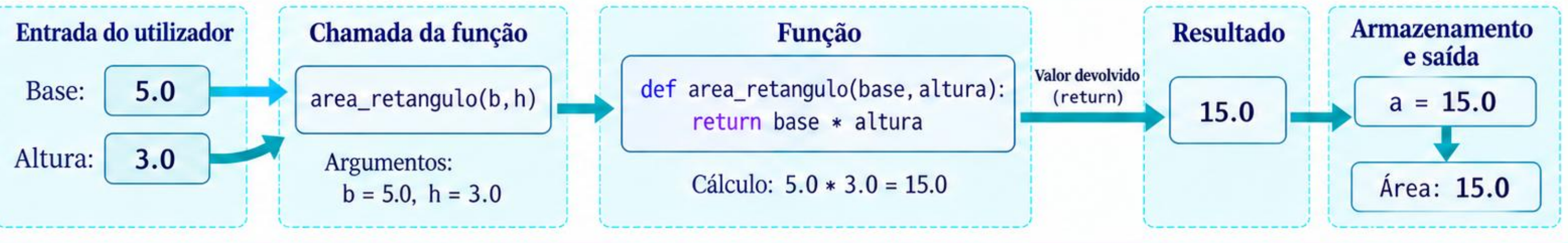
Dados de entrada:

O programa lê os valores introduzidos pelo utilizador com `input()` e converte-os com `float()`.

Resultado:

a função devolve o valor calculado, que é guardado na variável `a` e mostrado no ecrã com `print()`.

Fluxo de dados



FUNÇÕES, DECISÕES E CICLOS

Código em Python:

```
def mostrar_pares(limite):  
    for n in range(1, limite + 1):  
        if n % 2 == 0:  
            print(n)  
  
mostrar_pares(6)
```

← **Nível 0 (def)**
← **Nível 1 (for)**
← **Nível 2 (if)**
← **Nível 3 (print)**

Saída produzida:

2
4
6

A função `mostrar_pares` contém um ciclo, e o ciclo contém uma decisão. São examinados seis valores e impressos três valores (os pares).

Traçado da execução (limite = 6)				
Iteração	Valor de n	Condição $n \% 2 == 0$	Imprime?	Resultado
1	1	Falso	Não	—
2	2	Verdadeiro	Sim	2
3	3	Falso	Não	—
4	4	Verdadeiro	Sim	4
5	5	Falso	Não	—
6	6	Verdadeiro	Sim	6

Função:
bloco de código que pode ser reutilizado.

Ciclo (for):
repete um conjunto de instruções para cada valor.

Decisão (if):
executa instruções apenas quando uma condição é verdadeira.

Decomposição de um problema

Problema

Calcular a área e o perímetro de um retângulo.

Decomposição

O problema pode ser decomposto em três responsabilidades:

- 1 Obter a base e a altura do retângulo.
- 2 Calcular a área e o perímetro em funções distintas.
- 3 Apresentar os resultados.

Funções

Definirmos duas funções para realizar os cálculos.

Função da área

```
def area_retangulo(base, altura):  
    return base * altura
```

Função do perímetro

```
def perimetro_retangulo(base, altura):  
    return 2 * (base + altura)
```

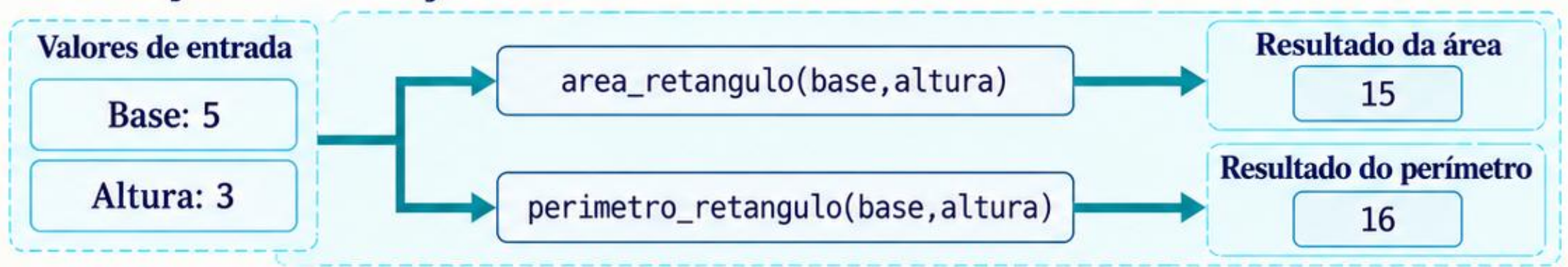
Exemplo

Para um retângulo com base 5 e altura 3:

Base	Altura	Área	Perímetro
5	3	15	16

Utilização das funções

O mesmo par de valores (base e altura) é fornecido a cada função.



Contratos das funções

Cada função recebe dois valores (base e altura) e devolve um resultado.

- `area_retangulo(base, altura)` devolve `base * altura`.
- `perimetro_retangulo(base, altura)` devolve `2 * (base + altura)`.

Notas

- A base e a altura representam as dimensões do retângulo e devem ser valores numéricos não negativos.
- Ainda não é apresentado o código principal que obtém os valores e apresenta os resultados.

Contrato de uma função

```
def area_retangulo(base, altura):  
    """Devolve a área; dimensões não negativas."""  
    return base * altura
```

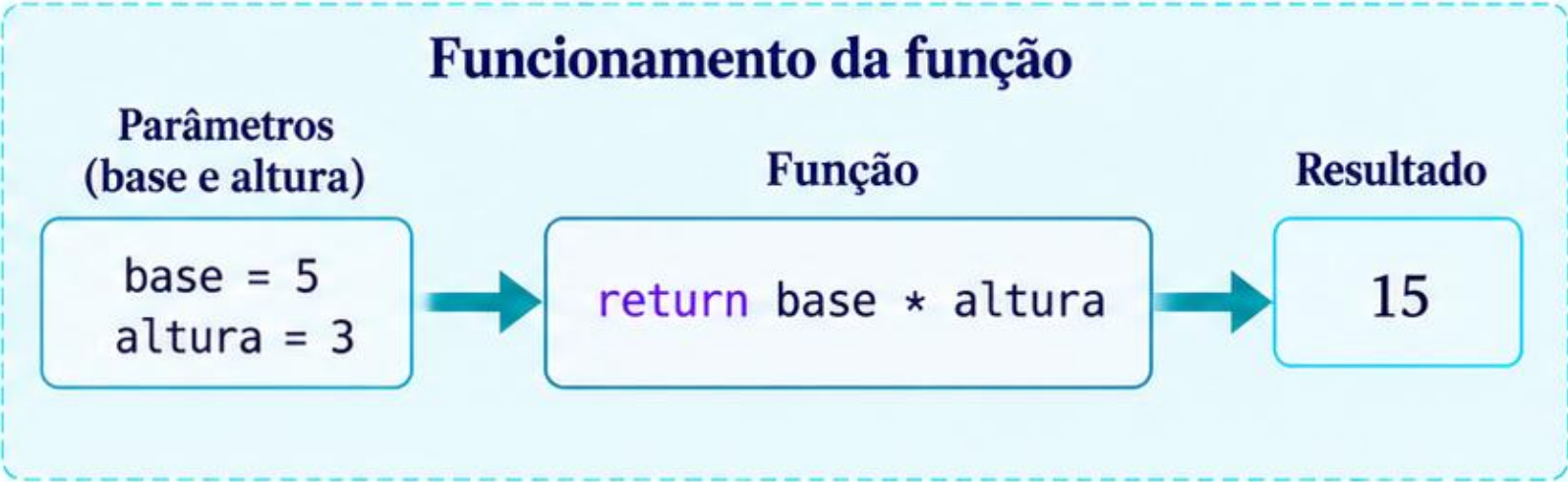
Objetivo:
Devolver a área de um retângulo.

Parâmetros:
base e altura são valores numéricos não negativos.

Resultado:
Devolve um número igual a `base * altura`.

Efeitos laterais:
Não lê nem imprime valores.

Chamada	Valores dos parâmetros	Resultado
<code>area_retangulo(5, 3)</code>	<code>base = 5</code> <code>altura = 3</code>	15
<code>area_retangulo(0, 3)</code>	<code>base = 0</code> <code>altura = 3</code>	0



Pré-condição:
Assume-se que base e altura são valores numéricos não negativos. Esta função não valida as entradas.

Valor devolvido:
Se a função terminar, devolve o valor da expressão `base * altura`.

Aplicação: média de três valores

```
def media_tres(a, b, c):  
    return (a + b + c) / 3
```

```
m = media_tres(10, 14, 18)  
print(m)
```

Parâmetros:

são os nomes das variáveis que recebem os valores numéricos.

Parênteses:

são necessários para dividir a soma total por 3.

Valores de teste	Valor devolvido
media_tres(0, 0, 0)	0.0
media_tres(12, 12, 12)	12.0
media_tres(10, 14, 18)	14.0

Como funciona a chamada da função

Valores de entrada

a = 10

b = 14

c = 18

Soma dos valores

$10 + 14 + 18$
 $= 42$

Divisão por 3

$42 / 3$
 $= 14.0$

Valor devolvido

14.0

Armazenado em

m = 14.0

A função recebe três valores numéricos através dos parâmetros **a**, **b** e **c**.

Os parênteses garantem que a soma total (**a + b + c**) é dividida por 3, e não apenas o último valor.

A função devolve a média dos três valores e o resultado é armazenado na variável **m**.

Aplicação: conversão de temperatura

Código em Python

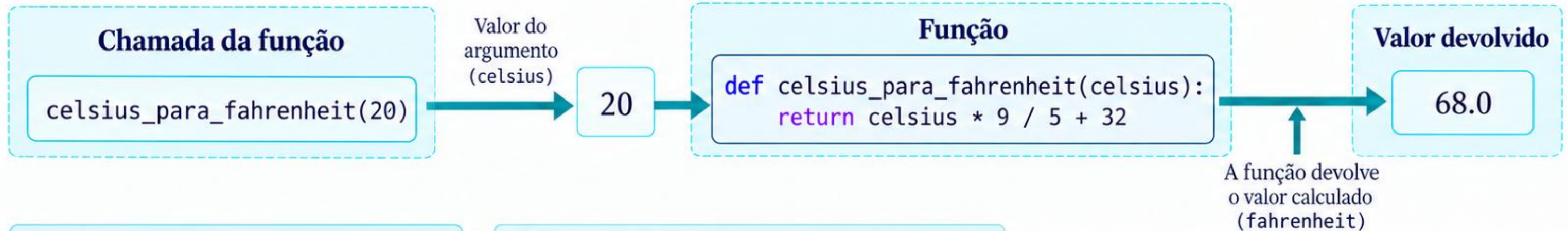
```
def celsius_para_fahrenheit(celsius):  
    return celsius * 9 / 5 + 32  
  
f = celsius_para_fahrenheit(20)  
print(f)
```

Função:

converte uma temperatura de graus Celsius para graus Fahrenheit e devolve o valor numérico calculado.

Fórmula:

$$F = C * \frac{9}{5} + 32$$



Saída no ecrã

68.0

Alguns testes

C (Celsius)	F (Fahrenheit)
-40	-40.0
0	32.0
20	68.0
100	212.0

A função devolve um número

A instrução **return** envia o valor numérico calculado para o código que chamou a função, que decide como apresentar esse valor.

Exercício: área e perímetro

Enunciado

Escreva um programa que leia a base e a altura de um retângulo (valores numéricos não negativos), defina as funções `area_retangulo(base, altura)` e `perimetro_retangulo(base, altura)`, ambas devolvendo um resultado numérico, chame as duas funções e mostre os resultados.

Considere que os valores de entrada são válidos. O valor zero é aceite como valor limite no cálculo.

Estrutura do programa

- 1 Ler a base e a altura.
- 2 Chamar `area_retangulo(base, altura)` e mostrar o resultado.
- 3 Chamar `perimetro_retangulo(base, altura)` e mostrar o resultado.

Fórmulas

A área de um retângulo é dada por:

$$\text{área} = \text{base} * \text{altura}$$

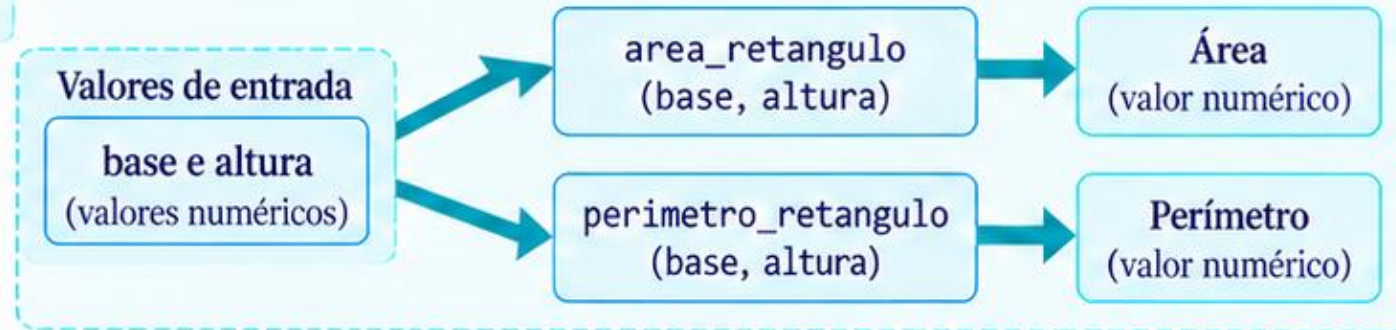
O perímetro de um retângulo é dado por:

$$\text{perímetro} = 2 * (\text{base} + \text{altura})$$

Exemplos de execução

Base	Altura	Área	Perímetro
5	3	15	16
4	4	16	16
0	3	0	6

Chamada das funções



Questões

- Quais são os parâmetros de cada função?
- Que valor é devolvido por cada função?
- Em que parte do programa são lidos os valores de entrada e em que parte são apresentados os resultados?

Notas

A função `area_retangulo` recebe a base e a altura e devolve a área do retângulo.

A função `perimetro_retangulo` recebe a base e a altura e devolve o perímetro do retângulo.

Solução: área e perímetro

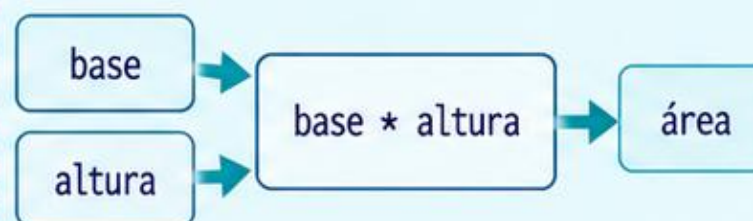
Código em Python

```
def area_retangulo(base, altura):  
    return base * altura  
  
def perimetro_retangulo(base, altura):  
    return 2 * (base + altura)  
  
b = float(input('Base: '))  
h = float(input('Altura: '))  
print('Área:', area_retangulo(b, h))  
print('Perímetro:', perimetro_retangulo(b, h))
```

Função da área

Calcula a área de um retângulo com base e altura dadas.

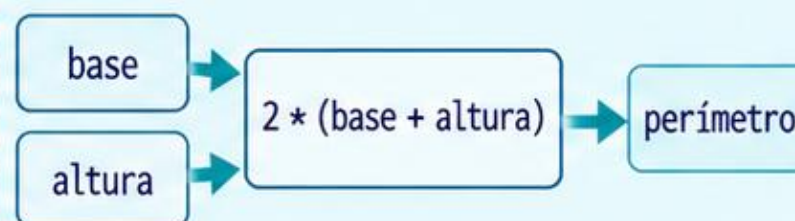
area_retangulo(base, altura)



Função do perímetro

Calcula o perímetro de um retângulo com base e altura dadas.

perimetro_retangulo(base, altura)



Exemplo de execução

```
Base: 5  
Altura: 3  
Área: 15.0  
Perímetro: 16.0
```

As funções realizam apenas os cálculos. A leitura dos dados e a apresentação dos resultados é feita fora das funções.

Assume-se que as dimensões (base e altura) são não negativas.

Testes de pequenas funções

Funções definidas anteriormente

As funções `area_retangulo` e `perimetro_retangulo` foram definidas anteriormente. Aqui vamos testá-las com diferentes valores.

Dados de teste

Vamos usar diferentes valores para a base e a altura e comparar os resultados obtidos com os esperados.

Caso de teste	Base	Altura	Área esperada	Perímetro esperado
Retângulo	5	3	15	16
Quadrado	4	4	16	16
Base nula	0	3	0	6
Valores decimais	2.5	4	10.0	13.0

Exemplos de chamadas de teste

```
print(area_retangulo(5,3))
```

Saída no ecrã

15

```
print(perimetro_retangulo(5,3))
```

Saída no ecrã

16

Como realizar os testes

- 1 Chamar a função diretamente com os valores de teste (sem usar input).
- 2 Ver o valor devolvido no ecrã.
- 3 Comparar o valor obtido com o valor esperado da tabela.

Objetivo

Verificar se as funções produzem os resultados correctos para diferentes casos de teste, incluindo um caso com zero e um caso com valores decimais.

Síntese — Funções I

Código em Python

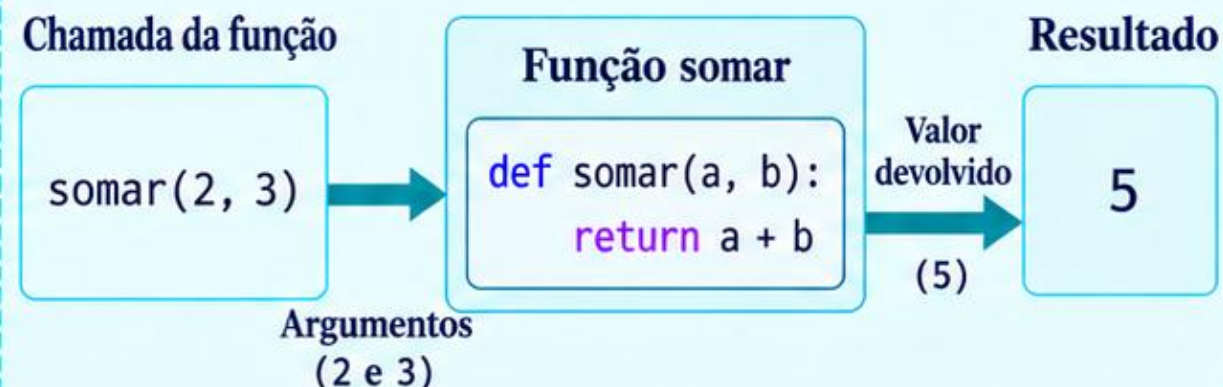
```
def somar(a, b):  
    return a + b
```

```
x = somar(2, 3)  
print(x * 2)
```

O que faz cada elemento

<code>def</code>	define uma função.
<code>()</code>	os parênteses são usados para chamar a função.
Parâmetros	recebem os argumentos fornecidos na chamada da função.
<code>return</code>	fornece um valor e termina a chamada da função.
<code>print</code>	mostra o valor no ecrã.
Funções pequenas	ajudam a separar responsabilidades no código.

Como funciona a chamada da função



Execução passo a passo

- 1 A chamada `somar(2, 3)` envia os argumentos 2 e 3 para a função.
- 2 A função calcula $2 + 3$ e devolve 5 através de `return`.
- 3 O valor devolvido é usado em $x * 2$, ou seja, $5 * 2$, que resulta em 10.
- 4 A instrução `print` mostra 10 no ecrã.

Resultado no ecrã

10

O que vem a seguir

Na próxima aula, vamos explorar o âmbito das variáveis, a composição de funções, a organização do código e os testes.